

BornoRMS — Restaurant ERP/POS

Complete End-to-End Business-Workflow & Production-Readiness Audit

Single-branch restaurant · Dine-in · Takeaway · Delivery · Cash / Card / Mobile Banking | Audit date: 15 June 2026

PRODUCTION-READY SCORE

58 / 100

RECOMMENDATION

Ready with Major Fixes

Executive Summary

BornoRMS is a genuinely mature build for its core — not demo-ware. The order lifecycle, POS billing math, split/partial payments, optimistic-concurrency protection, recipe-based inventory deduction (verified wired), the kitchen board, table/session management, role-based security, and an immutable financial audit log are all **really implemented and wired**. This is well above what these audits typically find.

It is **not** safe to run a full multi-channel restaurant today because of a cluster of money- and stock-integrity gaps: **payment force-completes an order and bypasses the kitchen workflow entirely** (a paid order jumps straight to *Completed* and disappears from the kitchen board — combined with no auto-printed ticket, the kitchen can be told nothing about food that was paid for); refunds and line-voids never reverse inventory; refunds/voids never post a ledger reversal (and an already-imported sale is not corrected); accounting is single-entry with **manual** posting; kitchen tickets (KOT) are **not auto-printed**; and **delivery has no logistics at all**. For a **dine-in + takeaway, cash/card** operation with disciplined staff it is workable, but the kitchen-bypass and KOT gaps must be fixed first. For delivery, accurate food-cost/profit, or hands-off accounting, it is not ready.

Critical Issues — revenue / stock / accounting / kitchen

1. Payment bypasses the kitchen workflow — an order can be fully processed without ever being confirmed. `Order.RecomputePaymentState()` (`Order.cs:451-456`) force-sets `Status = Completed` the moment an order is fully paid, jumping over the entire guarded state machine (`Placed` → `Confirmed` → `Preparing` → `Ready` → `Served`). The settle/quick-pay handlers (`SettleOrderCommand.cs` , `AddPaymentCommand.cs` — whose own comment says "completes an order without a separate Confirm step") never check kitchen status before taking money. The kitchen board (`GetKitchenBoardQuery.cs`) shows only *Placed/Confirmed/Preparing/Ready*, so a paid order **instantly vanishes from the KDS**. In the normal counter/takeaway "pay-first" flow the kitchen can therefore be told **nothing** — especially since KOT is not auto-printed (see #6). Secondary damage: prep-time analytics are skewed (kitchen timestamps stay null), "Completed" means *paid* not *served*, and a paid order is locked (only a refund can undo it). → **Customer pays, food never cooked.**

2. Refund does not restore inventory. `RefundPaymentCommand` bumps the drawer cash-out and records the refund but performs **no stock restoration**. Stock reverses only on order *Cancel* (`ChangeOrderStatusCommand.cs:73-74`). Refunding a confirmed/served order permanently undercounts inventory. → Stock loss.

3. Post-confirm line-void leaves stock deducted. `VoidOrderLineCommand` deletes the line and writes a note only. Stock is deducted at `Confirmed` (`ChangeOrderStatusCommand.cs:71-72`), so a later line-void leaks ingredients. → Stock loss.

4. No GL reversal on void/refund. Only the drawer's `CashPaidOut` is adjusted; no `FinanceTransaction` reversal posts. If takings were already imported, `AccountedAtUtc` is never cleared, so prior-day books overstate income with no auto-correction. → Accounting mismatch.

5. Accounting is single-entry, manual posting. `FinanceTransaction` has no balancing counter-account and there is no automatic journal entry on order payment — cash takings are imported/keyed by a human. → Reconciliation depends on human accuracy.

6. Kitchen tickets are not auto-printed. `KitchenTicketDocument` exists, but no handler fires a KOT print on place/confirm; printing is manual from the Cash Counter UI. → Kitchen will miss orders in a rush.

7. Delivery is an enum, not a workflow. No Rider/Driver entity, no dispatch/assignment, no delivery-fee field, no tracking, no delivery reports. → Cannot operate delivery.

8. No real payment gateway. Only `MockPaymentGateway` ; card/MFS auth is simulated. Combined with no request idempotency key, retried settles can double-record a tender.

9. Negative stock allowed silently. Consumption permits negative on-hand with no hard stop and no oversell alert; the availability check is advisory only. → Stock can drift deeply negative unnoticed.

Missing Features — prioritized

Critical: delivery logistics; auto-print KOT; inventory reversal on refund & post-confirm line-void; automatic/reversible accounting posting (or at least GL reversal on void/refund); real payment-gateway integration with idempotency.

High: Purchase Orders + PO→GRN matching; supplier payables/aging/statements; purchase returns; profit report (needs product COGS mapping — `Product` has no cost field); VAT/Tax report; cashier/shift & EOD-close reports.

Medium: coupon/promo system; item-level discounts; advance payments/deposits; recipe versioning; drawer reconciliation + petty-cash/float; barcode/SKU scan & fast POS product search.

Low: multi-level & cross-dimension unit conversion; formal stock-count sessions / cycle counts / bin tracking; reservation depth (time windows, waitlist, overbooking).

Workflow Gaps

- **Refund flow** stops at money; doesn't touch inventory or GL — staff must separately cancel the order to fix stock.
- **Delivery flow** ends at order creation; no dispatch → deliver → settle loop.
- **Kitchen flow** has no automatic ticket emission.

- **No order-acceptance step on any channel.** POS/Waiter/QR all create the order as `Placed` ; nothing confirms it. The only `Confirm()` is a side-effect of kitchen staff hitting "Start" on the KDS (`AdvanceKitchenOrderCommand` → `BeginPreparing`). So the kitchen is the *implicit* confirmer, and QR/online orders get no acknowledgement back to the customer. See the design section below.
- **Purchase flow** is GRN-only: no PO, no return, no payable — "Pay Supplier" has nothing to settle against.
- **Accounting close** is import-driven and one-directional; refunds after import are not back-corrected.

Verified working: dine-in loop (table → session → order → confirm → kitchen board → serve → settle → table freed), takeaway, order state machine with illegal-transition rejection, split/partial payment, manager-approved void/refund with audit snapshots, recipe explosion + idempotent stock deduction on confirm, stock reversal on cancel.

Order Confirmation & the Real-World Workflow Model

The single root cause behind Critical #1 and the confirmation gap is that BornoRMS collapses two lifecycles into one. Every production restaurant POS (Toast, Square for Restaurants, Lightspeed, Petpooja) keeps them **independent**:

- **Order / kitchen status:** Open → Fired/In-Kitchen → Ready → Served → Closed
- **Payment status:** Unpaid → Partially paid → Paid

An order is **Closed/Completed only when BOTH are done — food delivered *and* paid**. Payment never force-completes an order; firing to the kitchen never marks it paid. BornoRMS instead lets payment drive `Order.Status = Completed`, which is why a paid order vanishes from the kitchen board.

Who "confirms," per service model (real world)

Channel	Who confirms / fires	What confirmation means
Full-service dine-in (waiter)	Waiter hits "Send / Fire"	The fire action <i>is</i> the confirmation — an accepted, committed kitchen ticket; KOT auto-prints instantly. Bill & payment come last, separately.
Quick-service / takeaway (pay-first)	The payment event auto-fires the ticket	Paying does <i>not</i> close the order — it moves to <i>In-Kitchen</i> and stays on the KDS until the food is handed over (kitchen bumps "Complete"). "Paid" ≠ "fulfilled".
Online / QR / delivery	Manager/cashier taps "Accept" (or auto-accept rule)	Accept sends a confirmation + prep-ETA <i>back to the customer</i> , then fires to the kitchen. Never silently dropped into the queue.

The one-line model: Front-of-house *accepts/fires* the order (that is the confirm) · the kitchen *cooks* it · the cashier *collects* payment independently · the order closes only when served **and** paid.

Real practice vs. BornoRMS today

Real-restaurant practice	BornoRMS today
Order acceptance is explicit at point of entry (fire/accept)	No acceptance step — order sits Placed ; kitchen implicitly "confirms" by starting to cook
KOT auto-prints/displays the instant the order is fired	KOT not auto-printed; KDS only, manual print
Payment and fulfilment are independent axes	Conflated — payment force-sets Completed
A paid order stays in the kitchen until food is handed over	Paid order vanishes from the KDS
Online/QR sends an acceptance + ETA back to the customer	No acknowledgement loop
Order closes only when served and paid	Closes on payment alone, regardless of whether it was cooked

Correct design for BornoRMS

1. **Auto-Confirmed on placement for QR & Waiter** (an accepted ticket immediately), or an explicit **"Accept"** action on the KDS for online orders, distinct from "Start cooking".
2. **Keep kitchen status orthogonal to `IsPaid`**. Payment sets `PaymentStatus = Paid` and nothing else — it must **not** touch `Order.Status`.

3. **Completed** requires **Served** **AND** **Paid**, not payment alone.
4. **Auto-fire the KOT** to the kitchen printer/station the moment an order is accepted/fired.
5. For QR, send an **acceptance + prep-ETA** back to the customer.

This reframes Phase 1 of the roadmap: the fix is not "add a confirm button" — it is to **separate the payment axis from the fulfilment axis**, the model every production restaurant POS uses.

Security Issues

- **Overall: solid.** Consistent policy-based authorization (`Staff` , `Admin` , `Kitchen` , `WaiterFloor` , `CanDiscount` , `CanVoid` , `CanSettle`) on Blazor pages *and* server-side `PermissionGuard.Require( ... )` backstops in handlers — not just UI hiding. Three auth mechanisms correctly segregated (customer OTP, staff JWT, staff cookie).
- **Manager approval is self-approvable** — a manager voiding/refunding their own transaction needs no second person. Fraud opportunity; consider dual-control + per-user void/refund reporting.
- **No per-cashier discount cap / abuse tracking** — only a single global high-discount threshold (default 20%).
- **No payment idempotency key** — retried settle could double-record; concurrency guarded by `RowVersion` but not request-level dedup.
- `/tables` API intentionally public (customer QR seating), read-only — acceptable.
- Rotate the seeded super-admin `admin@bornobit.local` / `ChangeMe!2026` before go-live.

Performance Issues

- Kitchen/dashboard real-time is hybrid: SignalR push + a 5-second polling safety-net (to catch API-process events) — functional, but a latency floor and constant query load that grows with volume.
- **No load-test evidence** for 20/50/100 concurrent users. **LocalDB is a dev database — not production-grade**; a real SQL Server is required.
- Cash-counter/board queries join on Payments in SQL — index-review before high volume. No caching layer.

Data Integrity Risks

- `AccountedAtUtc` not cleared on refund → double-count / overstated income across the import boundary.
- Stock under-count from refund / post-confirm line-void without reversal.
- No request idempotency → double payment/transaction on retry or double-click.
- Single-entry accounting → no structural debit=credit guarantee; manual-entry errors uncaught.
- LocalDB + Windows auth → single machine, no real HA/backup; both API and Web auto-migrate on boot (race).

Production Readiness Checklist

Module	Status	Note
Restaurant / billing settings (VAT, service, currency, rounding)	✓	Defaults applied at settle
Users / roles / permissions / authorization	✓	Policy + handler guards; rotate seed admin
Menu (categories, products, variants)	✓	No barcode/SKU
Combos / add-ons	⚠	Variants yes; combo/add-on depth unverified

Recipe management	⚠	Works; no versioning/costing
Inventory ledger + deduction on sale	✓	Recipe + direct-stock, idempotent, reverses on cancel
Inventory reversal on refund / line-void	✗	Stock leak
Negative-stock control	⚠	Allowed silently; alerts only
Suppliers	⚠	Entity only; no payables/statements
Purchase Orders	✗	GRN-only
Goods receiving (GRN)	✓	Posts stock with unit conversion
Purchase returns	✗	Missing
Supplier payments / payables	✗	Missing
Tables + states + sessions + reservations	✓	Reservations lightweight
Dine-in workflow	✓	Full loop verified
Takeaway workflow	✓	
Delivery workflow	✗	Enum only; no logistics
POS counter (calc, discount, tax, service, split)	✓	No barcode/coupon
Order lifecycle / state machine	✗	Forward transitions guarded, but payment force-jumps to Completed, bypassing all kitchen states
KDS (real-time board)	✓	Hybrid SignalR + 5s poll
Printing — receipts	✓	QuestPDF + SignalR print agent
Printing — KOT auto-emit	✗	Manual only
Payments (cash/card/mobile, split, partial)	✓	Mock gateway; no advance/idempotency
Refund / void	⚠	Money correct; no stock/GL reversal
Discounts	⚠	Order-level only; no coupon; self-approvable
Cash drawer open/close + variance	⚠	Variance computed, not reconciled to GL
Reporting — sales/category/hour/revenue/staff	✓	
Reporting — profit	✗	No COGS→order mapping
Reporting — VAT / cashier / EOD close	✗	Missing
Accounting — auto-posting / double-entry	✗	Single-entry, manual
Financial audit trail	✓	Immutable, before/after snapshots
Concurrency protection	✓	RowVersion + retries; Payment.Id fix landed
Request idempotency	✗	No server-side key
Performance / load tested	✗	No evidence; LocalDB not prod-grade

Appendix A — Competitive Gap: Waiter Console vs. Petpooja Captain App

BornoRMS's Waiter console is **strong on table/session management** — at rough parity with Petpooja's Captain app on most floor operations, and ahead on the built-in customer call-bell. It falls behind on **kitchen-firing mechanics** (auto-KOT, incremental round-KOTs, per-station tickets), **item-level detail** (modifiers/add-ons/per-item notes), and **handheld/offline ergonomics**. Grounded in

`WaiterOrders.razor.cs` + `GetWaiterDashboardQuery.cs`.

Capability	Petpooja Captain	BornoRMS Waiter page	Verdict
Live floor / table status	✅ color-coded map	✅ Floor tab, status filter, "mine only"	Parity
Open table + covers (PAX)	✅	✅ <code>OpenSessionCommand</code> + guest dialog, auto-jump to order	Parity
Item search / favorites	✅ search box, favorites	⚠ category filter only, no text search	Behind
Variations (full/half)	✅	✅ variant picker	Parity
Add-ons / modifiers	✅ modifier groups	❌ variants only	Gap
Per-item special instructions	✅ per line	⚠ domain supports line notes; waiter cart captures order-level notes only	Gap (UI)
Send/Fire KOT to kitchen	✅ one tap → auto-prints	❌ "Place" prints nothing; KOT is a manual PDF	Critical gap
Running / round KOTs (add more)	✅ new KOT of only new items	⚠ <code>UpdateWaiterOrderLinesCommand</code> replaces the line set; no incremental round-KOT	Gap
Per-station / department KOTs	✅ separate KOT per kitchen	⚠ lines carry <code>StationId</code> , KDS filters by it, but printed KOT is one combined ticket	Partial
Modify/cancel item + reason + manager auth	✅	⚠ <code>VoidOrderLineCommand</code> exists (manager) but not surfaced on the waiter page	Gap
Coursing (fire starters now, mains later)	✅	❌ none	Gap
Transfer table / shift KOT	✅	✅ <code>MoveSessionTableCommand</code>	Parity
Merge tables	✅	✅ <code>MergeSessionsCommand</code>	Parity
Split table / bill	✅	✅ <code>SplitSessionCommand</code>	Parity
Transfer waiter / captain	✅	✅ <code>TransferSessionWaiterCommand</code>	Parity
Mark served	✅	✅ Ready tab → Serve / Serve-all	Parity
View bill / request bill	✅ captain can bill if permitted	✅ view bill + <code>RequestCashierSettlementCommand</code> (waiter requests, cashier settles)	Design choice
Customer call-bell (water/bill/waiter)	⚠ separate QR product	✅ Requests tab: Call Waiter / Bill / Water / Tissue	Ahead
Waiter sales attribution	✅ captain-wise report	✅ "my tables / my revenue today" widgets	Parity
Real-time sync	✅ cloud push	✅ SignalR dashboard hub (best-effort)	Parity

Offline mode	✅ queues offline, syncs later	❌ Blazor Server — dies without connection	Critical gap
Hardware	✅ native Android/iOS handheld	⚠ web (tablet browser; heavier, connection-bound)	Behind
Acceptance + ETA back to customer	✅ (online orders)	❌ none	Gap

Summary: On table & session management BornoRMS is at rough Captain-app parity (and ahead on the call-bell). On kitchen firing — auto-KOT, round-KOTs, per-station tickets, modifiers, offline — it is materially behind, and the missing auto-KOT is the same go-live blocker Critical #1 and #6 already flag, seen from the waiter's seat.

Appendix B — Competitive Gap Analysis vs. Petpooja (Whole System)

A like-for-like module count would mislead — Petpooja is a 10-year commercial SaaS with a large integration marketplace, and **multi-outlet / central-kitchen / franchise are intentionally out of scope** for BornoRMS (tenancy was deliberately stripped; single-branch by design). The fair bar is parity on *core single-branch restaurant operations*, which BornoRMS largely reaches. Gaps below are split into genuine functional gaps, market-specific integrations, and the few correctness defects that matter more than any missing module.

B.1 — Whole-system scorecard

Module	Petpooja	BornoRMS	Gap
POS billing (split/partial/rounding/numpad)	✓ mature	✓ strong	Parity
Cash drawer / shift + variance	✓	✓	Parity
Table / session / floor management	✓	✓ merge/split/transfer	Parity
Waiter / captain app	✓ native, offline	⚠ web, no offline	Minor
Customer QR / own-brand online ordering	✓ (paid add-on)	✓ Next.js site (phone-OTP, menu, cart)	Parity (strength)
Menu (categories / variations)	✓	✓	Parity
Menu (add-ons / modifiers / combos)	✓	✗ variants only	Gap
Recipe / BOM + stock deduction	✓	✓ recipe explosion, idempotent	Parity
Inventory ledger / wastage / adjust	✓	✓ full ledger	Parity
Purchase Orders + GRN matching	✓	⚠ GRN only, no PO	Gap
Purchase returns	✓	✗	Gap
Supplier payables / statements	✓	✗ entity only	Gap
KDS (real-time board)	✓	✓ SignalR + poll	Parity
KOT auto-print / per-station / round-KOT	✓	✗ manual, combined	Critical gap
Payments — gateway (card / UPI / online link)	✓ + Petpooja Pay	✗ mock only	Critical gap
Discounts / coupons / happy-hours / loyalty	✓ rich	⚠ order-level %/flat only	Gap
CRM / loyalty / feedback / SMS-WhatsApp	✓	✗ none	Gap
Tax compliance (multi-slab, e-invoice)	✓ GST	⚠ single VAT% + service%	Gap
Accounting (auto-post, double-entry)	✓ + Tally sync	✗ single-entry, manual	Gap
Expense management from POS	✓	✗	Gap
Reports & analytics	✓ 200+	⚠ dashboards; no profit/VAT/COGS	Gap
Delivery logistics (rider / dispatch / fee)	✓ + logistics	✗ enum only	Critical gap
Aggregator integration (→ Foodpanda / Pathao)	✓ Zomato/Swiggy	✗	Gap (market)
Offline mode	✓	✗ Blazor Server	Critical gap

Multi-outlet / central kitchen / chain	✓	✗ single-branch by design	Out of scope
Hardware ecosystem (scale, caller-ID, multi-printer)	✓	⚠ printer agent only	Gap
Owner mobile app (remote monitoring)	✓	✗	Gap
RBAC / roles / permissions	✓	✓ policy + handler guards, DB-driven nav	Parity

B.2 — Cashier vs. Petpooja Billing (detail)

Grounded in `Pos.razor.cs` , `PaymentDialog.razor` , `CashCounter.razor.cs` . Note the cashier is split across two screens — the POS terminal (take-order + quick-pay) and the Cash Counter (board / EOD / session settlement); Petpooja unifies billing in one screen.

Capability	Petpooja Billing	BornoRMS	Verdict
Touch numpad billing	✓	✓ quick keys (Remaining/50+/100+/500+)	Parity
Split payment (multi-tender)	✓	✓ staged tenders	Parity
Partial / part payment	✓	✓ "Take Partial"	Parity
Cash change calc	✓	✓ overpay → change	Parity
Payment modes	✓ cash/card/UPI + online links	△ cash/card/mobile, mock gateway, no UPI/link	Behind
Round off	✓	✓ Floor/Ceil	Parity
Discounts %/flat	✓ + item-level + coupons + loyalty	△ order-level + manager threshold	Behind
Tax/GST breakup on bill	✓ multi-slab, HSN	△ single VAT% + service%	Behind
Reprint bill	✓	✓	Parity
Receipt auto-print on settle	✓	✓ fire-and-forget	Parity
KOT auto-print	✓	✗ manual	Critical gap
Void / modify bill + reason + manager	✓	✓ manager-gated + audit	Parity
Bill on hold / running orders	✓	✓ chips, every edit auto-persisted	Parity
NC / complimentary / staff-meal bill	✓	✗	Gap
Cash drawer / shift open-close	✓	✓ + variance	Parity
Expense / cash pay-out from POS	✓	✗	Gap
Day-end / settlement (Z) report	✓	△ daily summary + ledger, no Z doc	Partial
Customer / CRM capture	✓ phone/GST + loyalty	△ phone/name/address only	Behind
Multi-terminal real-time	✓	✓ SignalR + busy-guard	Parity
Double-charge concurrency safety	△	✓ drops staged tenders on conflict; Payment.Id fix	Ahead
Idempotency key on settle	partial	✗ recovery only, no key	Behind
Offline billing	✓	✗ Blazor Server	Critical gap

B.3 — Reading the gaps

Entire categories missing: online-payment/gateway · delivery logistics · aggregator integration (BD: Foodpanda/Pathao) · CRM/loyalty/marketing · procurement depth (PO/returns/payables) · expense

management · double-entry accounting · analytics depth (profit/VAT/Z) · offline resilience · multi-slab tax compliance.

Present but shallow: menu add-ons/combos · coupons/happy-hours/item-level discounts · kitchen firing (auto-KOT, round-KOTs, per-station) · reporting (no profit/VAT/cashier-Z).

Genuinely competitive (real parity): POS billing + double-charge concurrency hygiene · table/session management (ahead on the customer call-bell) · inventory + recipe deduction · own-brand QR/web ordering (Petpooja charges extra) · RBAC + DB-driven navigation.

Correctness defects (matter more than any missing module): payment force-completes the order and bypasses the kitchen · refund/line-void don't reverse inventory · void/refund post no GL reversal and `AccountedAtUtc` isn't cleared. Fix these before chasing feature parity.

B.4 — Prioritized gap closure (P0–P3)

Priority	Gap cluster	Roadmap
P0 — blockers	Auto-KOT · settlement-bypass · refund inventory/GL reversal · real payment gateway	Phase 1
P1 — channel / money	Delivery logistics · profit/VAT/EOD reports · online payment for QR	Phase 2
P1 — experience	Add-ons/modifiers · coupons/loyalty · round-KOT/per-station · offline till + captain	Phase 2b
P2 — back-office	PO/returns/payables · expense mgmt · double-entry/Tally export · multi-slab tax	Phase 3
P3 — growth	Aggregator (Foodpanda/Pathao) · CRM/feedback/WhatsApp · owner app · hardware ecosystem	Phase 4

Bottom line: BornoRMS reaches operational parity with Petpooja on the core dine-in loop. The gaps cluster in money completeness (gateway, accounting, tax, profit), kitchen firing (auto-KOT), growth modules (delivery, aggregators, CRM), and resilience (offline) — and under all of it sit three correctness defects to fix first.

Final Verdict

Can this software run a real restaurant today? Not yet — even for dine-in + takeaway, the **payment-bypasses-kitchen** behaviour plus no auto-KOT means a paid order can reach no one in the kitchen, which is a go-live blocker on its own. The POS billing math and the manual kitchen loop are individually trustworthy, but they are not wired together safely. The blockers are: decouple "paid" from "fulfilled" (stop payment from force-completing an order), auto-print the KOT, stock-reversal correctness on refund/void, accounting automation/reversal, delivery logistics, and a real payment gateway. With those fixed, dine-in + takeaway on cash/card is viable; delivery and hands-off accounting need more.

Go-live roadmap (priority order)

Phase 1 — Integrity blockers (before any go-live): (1) **decouple payment from fulfilment** — stop `RecomputePaymentState()` force-completing the order; keep kitchen status independent of `IsPaid` so a paid order stays visible to the kitchen until actually served (track "paid" and "served" separately); (2) auto-emit KOT on confirm/place via the existing `PrintHub` with job dedup; (3) reverse inventory on refund & post-confirm line-void (mirror `OrderStockSync.TryReverseAsync`); (4) post a `FinanceTransaction` reversal on void/refund & clear/re-evaluate `AccountedAtUtc`; (5) add a server-side idempotency key to payment/settle commands; (6) rotate seed super-admin; move off LocalDB to real SQL Server with backups.

Phase 2 — Channel + money completeness (before delivery / card go-live): (7) delivery subsystem (rider, dispatch, delivery fee, tracking, report); (8) real payment-gateway integration (card + bKash/Nagad) replacing the mock; (9) profit reporting (roll `InventoryItem.AvgCost` up via recipe → COGS-per-order) plus VAT and cashier/EOD reports.

Phase 2b — Waiter/captain parity (Petpooja gap closure — see Appendix A): (10) **incremental round-KOTs** — adding to a running order fires a KOT of only the new items, and **split the KOT per kitchen station**; (11) **per-item modifiers/add-ons + per-line special-instruction notes** in the waiter cart, and surface `VoidOrderLineCommand` (modify/cancel item with reason + manager auth) on the waiter page; (12) **offline-capable order capture** (queue + sync) so the floor survives WiFi drops; (13) item search/favorites in the take-order grid; optional coursing (hold/fire).

Phase 3 — Procurement + controls: (14) Purchase Orders + PO→GRN matching, purchase returns, supplier payables/statements/payments; (15) discount controls (item-level + coupon, per-cashier caps, dual-control + frequency reporting on manager void/refund); (16) drawer reconciliation to GL, petty-cash/float in-out, variance escalation.

Phase 4 — Hardening / depth: (17) recipe versioning + costing; formal stock-count sessions; multi-level unit conversion; reservation depth; load/concurrency testing to the 100-user target; dashboard caching.